

III. Murakkab Tarmoq Ssenariylari va Xatoliklar Bilan Ishlash

Ushbu bo'limda biz tarmoq dasturlarining barqarorligini (Robustness) ta'minlash, ma'lumotlarni strukturalangan holda uzatish va xavfsizlik tahdidlaridan himoyalaniшни o'rganamiz.

3.1. Blocking vs Non-blocking Sockets: Dastur qotib qolishining oldini olish

Soketlar sukut bo'yicha **Blocking** (to'xtatib turuvchi) rejimida ishlaydi. Bu degani, `accept()` yoki `recv()` metodlari chaqirilganda, operatsiya tugallanmaguncha yoki ma'lumot kelmaguncha dasturning butun ish oqimi to'xtab qoladi.

- **Blocking Sockets:** Dastur ma'lumot kelishini "kutadi". Agar tarmoqda muammo bo'lsa yoki qarshi tomon xabar yubormasa, dastur javob bermay qoladi (freeze).
 - **Non-blocking Sockets:** `s.setblocking(False)` metodi orqali yoqiladi. Bunda metodlar kutib turmaydi; agar ma'lumot tayyor bo'lmasa, darhol xatolik qaytaradi. Bu ko'p sonli ulanishlarni bitta oqimda boshqarish imkonini beradi.
-

3.2. Timeouts va Reconnect: Tizim barqarorligi

Tarmoq ulanishi istalgan vaqtda uzilishi mumkin. Dastur bu holatda "cheksiz kutish" rejimiga tushib qolmasligi uchun **Timeout** mexanizmi kerak.

- `settimeout(value)` : Soket kutish vaqtini (masalan, 5 sekund) belgilaydi. Agar belgilangan vaqt ichida javob kelmasa, `socket.timeout` xatoligi yuz beradi.
- **Reconnect (Qayta ulanish):** Agar `connect()` xato bersa, `while` sikli ichida ma'lum vaqt oralig'ida qayta urinish algoritmini tuzish lozim.

```
import socket
```

```
s = socket.socket()
s.settimeout(5.0) # 5 sekund kutadi
```

```
try:
    s.connect(('127.0.0.1', 5000))
except socket.timeout:
    print("Xato: Ulanish vaqti tugadi (Timeout)!")
except ConnectionRefusedError:
    print("Xato: Server faol emas!")
finally:
    s.close() # Resursni bo'shatish
```

3.3. JSON Data Exchange: Murakkab ma'lumotlarni uzatish

Oddiy matnlardan ko'ra, lug'atlar (dict) va ro'yxatlar (list) bilan ishlash qulayroq. Buning uchun **JSON (JavaScript Object Notation)** formati ishlatiladi.

Jarayon:

1. Python obyektini (lug'at) JSON satriga aylantirish: `json.dumps(data)` .
 2. Satrni baytga kodlash: `.encode('utf-8')` .
 3. Tarmoqdan kelgan baytni avval dekodlash, so'ng yana Python obyektiga qaytarish:
`json.loads(decoded_data)` .
-

3.4. Kiberxavfsizlik: "Denial of Service" (DoS) hujumlari

DoS hujumi — bu serverni juda ko'p so'rovlar bilan "bo'g'ib qo'yish" va uning qonuniy foydalanuvchilarga xizmat ko'rsatishini to'xtatishdir.

- **Soket darajasidagi xavf:** Agar serverda `listen(backlog)` navbati juda katta bo'lsa va ulanishlar yopilmasa, tizim xotirasi to'lib ketadi.
 - **Himoyalaniish metodlari:**
 1. **Rate Limiting:** Bitta IP-manzildan ulanishlar sonini cheklash.
 2. **Timeout o'rnatish:** Faol bo'lmagan (idle) klientlarni avtomatik uzib qo'yish.
 3. **Validation:** Kelayotgan ma'lumot hajmini `recv(bufsize)` orqali qat'iy nazorat qilish (masalan, 1024 baytdan oshirmaslik).
-

In []: